Using a builder pattern

```
    public static FFBuilder withFrequentFlyerNumber(String number) {
        return new FFBuilder(number);
    }

    public static class FFBuilder {

        private String frequentFlyerNumber;

        public FrequentFlyerBuilder(String frequentFlyerNumber) {
            this.frequentFlyerNumber = frequentFlyerNumber;
        }

        public FrequentFlyer named(String firstName, String lastName) {
            return new FrequentFlyer(frequentFlyerNumber,
                                     firstName,
                                     lastName);
        }
    }
}
```

Testing getters and setters

Experienced BDD (and TDD) practitioners generally avoid testing getter and setter methods in themselves. When you need getters, setters, and constructors, they're exercised by the unit tests that describe the behavior that requires them. Remember, your unit tests are executable specifications and living documentation; they describe what the properties of an object are for and how they're used. If you test them explicitly, you're merely stating that your object needs a "name" or a "status" field, for example, but you're not explaining why or how they should be used.

You now have a builder that will allow you to write code like the following to instantiate a new `FrequentFlyer` object:

```
FrequentFlyer member = FrequentFlyer.withFrequentFlyerNumber("12345678")
                                    .named("Joe","Bloggs");
```

REFACTOR IF REQUIRED

Once you've got the test to pass, it's always useful to review your code and see if it can be improved in any way. This includes code-quality considerations such as removing duplicated code, simplifying over-complex algorithms, grouping related lines of code into appropriately named methods, and making sure the variables are well named. But you can also refactor your code from an API perspective: is the API as simple and intuitive as it should be? For example, the `FrequentFlyer` builder might be more readable like this:

```
FrequentFlyer member = FrequentFlyer.withFrequentFlyerNumber("12345678")
                                    .withFirstName("Joe")
                                    .andLastName("Bloggs");
```

You should also include the executable specifications themselves in the refactoring activity; for example, does the name of the unit test reflect the functionality and code being illustrated? Are the variable names clear and accurate? And so on.